

上节回顾

- **两级页表与多级页表：对页表本身采用分页管理**
- **基本分段管理：分段是具有逻辑意义的分隔**
 - **段地址是二维的，段表包含段开始地址和长度**
 - **段物理地址的计算**
- **分页与分段的区别：P130,P135**
- **段页式管理：先分段后分页**
 - **地址计算：先求页号与页内地址，由段号查段表找到页表，由页号查页表得到块号，最后得到物理地址**

虚拟存储器

- 常规存储器管理面临的问题：
 - 单个作业很大，要求的内存超过总容量
 - 大量作业要求进入，要求总量超过总容量，使得并发数量受限制
- 常规存储器管理方式的特征
 - 一次性：内存浪费，每个作业都是一次性全部装入内存
 - 驻留性：占用资源，不使用部分长期占用内存

虚拟存储器

- 局部性原理： 1968年， Denning.P提出
 - 程序执行时， 除了少部分的转移和过程调用指令外， 在大多数情况下仍是顺序执行的
 - 过程调用将会使程序的执行轨迹由一部分区域转至另一部分区域， 但经研究看出， 过程调用的深度在大多数情况下都不超过5
 - 程序中存在许多循环结构， 这些虽然只由少数指令构成， 但是它们将多次执行
 - 程序中还包括许多对数据结构的处理， 如对数组进行操作， 它们往往都局限于很小的范围内

虚拟存储器

■ 局限性又表现在下述两个方面：

- 时间局限性。如果程序中的某条指令一旦执行，则不久以后该指令可能再次执行；如果某数据被访问过，则不久以后该数据可能再次被访问。
- 时间局限性的典型是在程序中存在大量的循环操作。
- 空间局限性。一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也将被访问，即程序在一段时间内所访问的地址，可能集中在一定的范围之内
- 典型情况便是程序的顺序执行。

虚拟存储器

- 虚拟存储器定义：所谓虚拟存储器，是指具有请求调入功能和置换功能，能从逻辑上对内存容量加以扩充的一种存储器系统
 - 其逻辑容量由内存容量和外存容量之和所决定
 - 其运行速度接近于内存速度
 - 成本接近于外存
- 虚拟存储器的虚拟特性：
 - 对内存空间的虚拟，将大容量的外存（交换区）作为内存管理
 - 对进程地址空间进行虚拟，允许进程有几乎无限大的地址空间

虚拟存储器

- 虚拟存储器的实现：分页请求系统、请求分段系统
- 分页请求系统
 - 在基本分页系统上增加请求调页功能和页面置换功能
 - 发生缺页就请求调页
 - 内存不足，就换出暂时不运行的页面
 - 硬件支持：页表机制、缺页中断、地址变换
- 请求分段系统
 - 在基本分段系统上增加请求调段功能和分段置换功能
 - 调入即将运行的段和换出暂不运行的段
 - 硬件支持：段表机制、缺段中断、地址变换

虚拟存储器

■ 虚拟存储器的特征

- 多次性：一个进程被分为多次调入内存运行
- 对换性：允许进程在运行过程中换入、换出
- 虚拟性：从逻辑上扩充内存容量

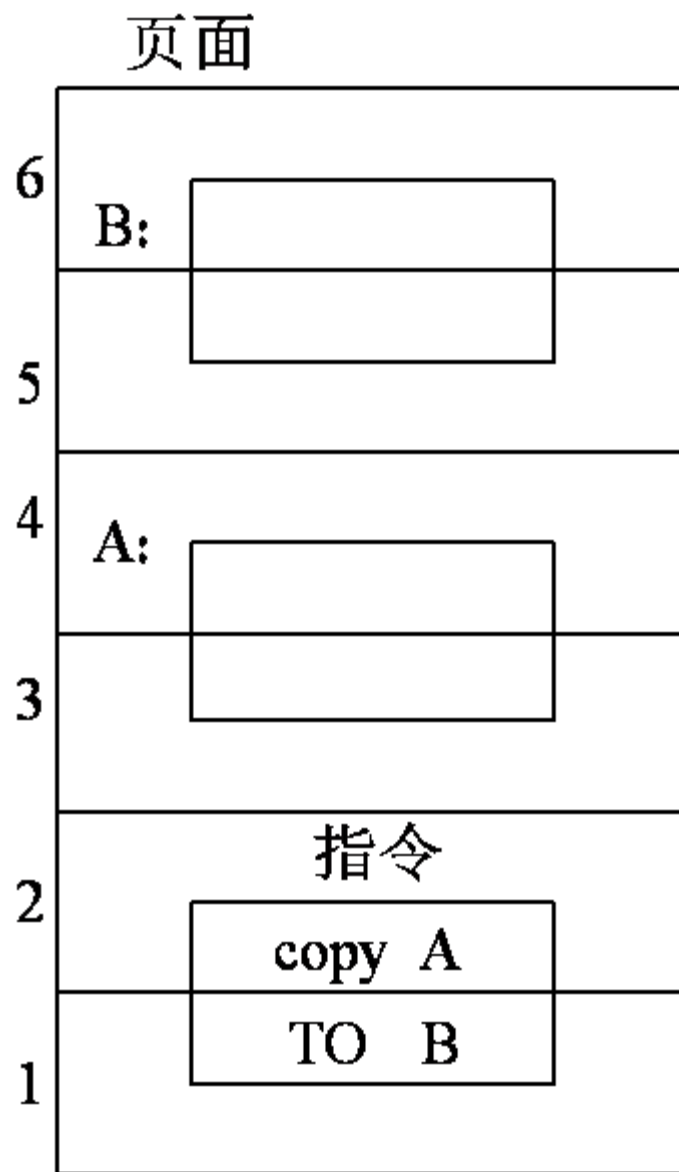
请求分页存储管理

- 请求分页中的硬件支持
 - 页表机制、缺页中断机构、地址变换机构
- 页表机制：纯分页的页表机制上增加若干项
 - 状态位：指示本页是否已经调入内存
 - 访问字段：记录本页在一段时间内访问的次数
 - 修改位：本页在调入内存后是否被修改过
 - 外存地址：本页在外存中的物理位置，一般是物理块号

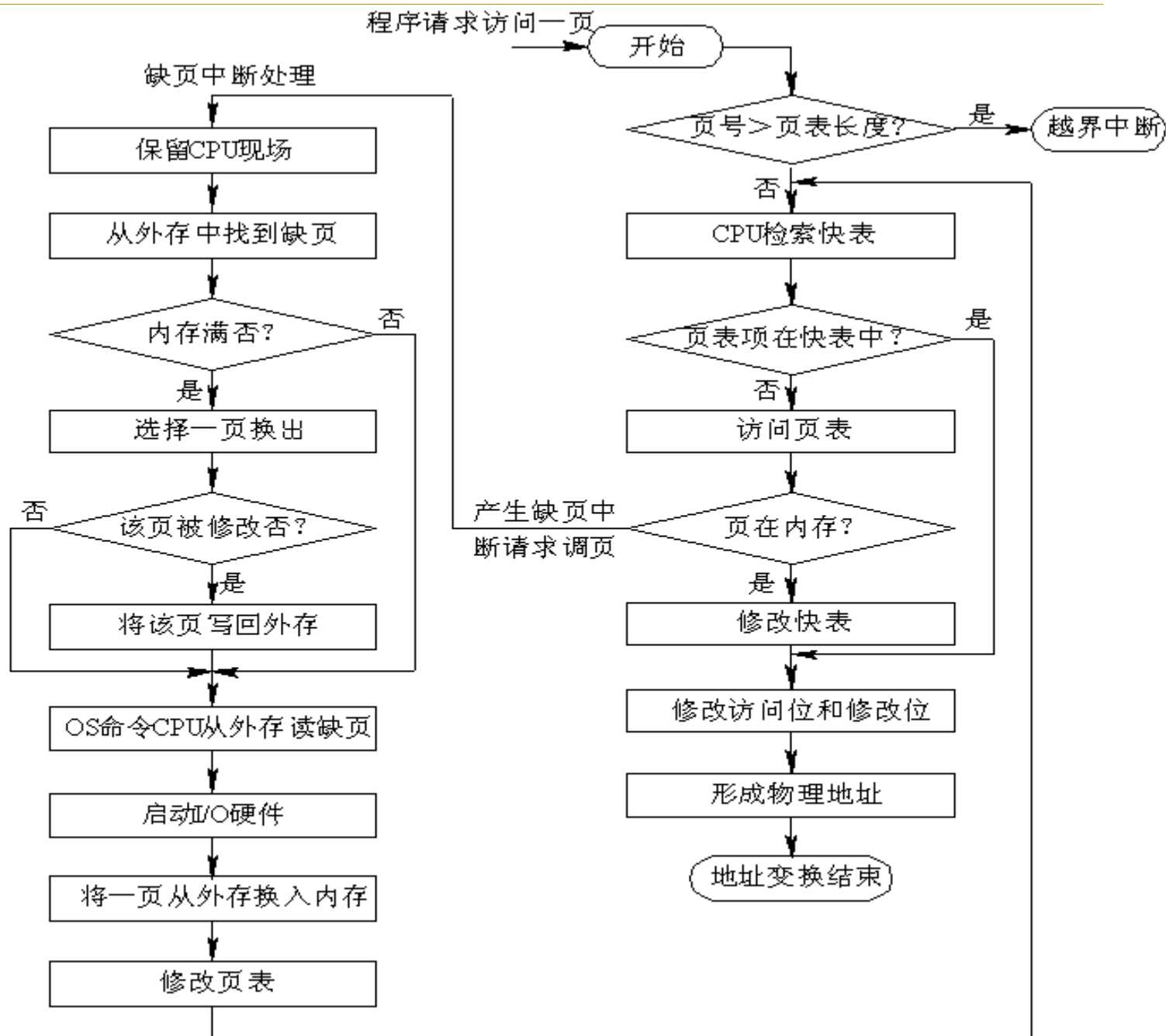
页号	物理块号	状态位 P	访问字段 A	修改位 M	外存地址
----	------	--------------	---------------	--------------	------

请求分页存储管理

- 缺页中断机构：当要访问的页面不在内存便产生缺页中断，以请求**OS**将所缺的页调入内存
- 缺页中断与一般中断的共同点：
 - 保存**CPU**现场
 - 分析中断原因
 - 缺页中断处理
 - 恢复**CPU**现场
- 缺页中断与一般中断的区别：
 - 缺页中断可能发生在指令执行过程中
 - 一条指令执行期间，可能发生多次缺页中断，最多可达**6**次



地址变换



请求分页存储管理

■ 地址变换过程

- ① 将逻辑地址划分为页号和页内地址
- ② 页号的越界检查
- ③ 从快表查找块号
- ④ 若找到，合并块号和块内地址得物理地址，修改访问位，若是写操作，设修改位为1
- ⑤ 若找不到，查页表
- ⑥ 若页面已在内存，将页号与块号调入快表，跳到步骤4
- ⑦ 若页面不在内存，调用缺页中断，跳到步骤6

请求分页存储管理

2 内存分配策略和分配算法

- 由于每个进程都不是一次性完全装入内存，而且内存的物理块数量有限，因此需要确定每个进程的物理块分配
 - 最小物理块数的确定
 - 物理块的分配策略
 - 物理块的分配算法

请求分页存储管理

■ 最小物理块数的确定

- 是指能保证进程正常运行所需的物理块最少数量
- 取决于指令的格式、功能和寻址方式
- 单地址指令+直接寻址，最少物理块数为**2**。一块存放指令，一块存放数据
- 系统支持间接寻址时，则至少要求有**3**个物理块
- 若多字节指令，指令本身跨两个页面，源地址和目标地址也可能跨两个页面，共涉及**6**个页面

请求分页存储管理

■ 物理块的三种分配策略

- ❑ 固定分配局部置换：每个进程的物理块数量固定，每次页面置换都在进程自身的物理块中进行
- ❑ 可变分配全局置换：**OS**保留一定的空闲物理块，根据进程需要动态分配。若无空闲物理块，页面置换选择任一进程的物理块，由算法决定
- ❑ 可变分配局部置换：分配同上，页面置换在进程自身的物理块中进行

■ 分配策略的评价指标：并发进程数、缺页率

- ❑ 并发进程数越多
- ❑ 每个进程的缺页率越低

请求分页存储管理

- 物理块分配算法
- 平均分配：所有空闲物理块平均分配给各个进程
 - 例系统有**100**个物理块，有**5**个进程在运行时，每个进程可分得**20**个物理块
 - 若其中一个进程大小为**200**页，只分配**20**个块，则缺页率很高
 - 若一个进程只有**10**页，则有**10**个物理块闲置未用
- 按比例分配：根据进程大小按比例分配物理块
 - n 个进程，每个进程的页面数为 S_i
 - 可用物理块总数为 m
 - 每个进程所能分到的物理块数为 b_i
 - b_i 取整且必须大于最小物理块数

$$S = \sum_{i=1}^n S_i$$

$$b_i = \frac{S_i}{S} \times m$$

请求分页存储管理

- 按优先权分配：对重要的、紧迫的作业分配较多的空间
 - 将空闲物理块分成两部分：一部分按比例地分配给各进程；另一部分则根据各进程的优先权，适当地增加其相应份额
- 例如**A\B\C\D**四个进程，分别需要**100、200、300、400**个物理块，现有**500**个物理块，则平均分配、比例分配和优先分配的结果

请求分页存储管理

调页策略

■ 何时调入页面

- 预调入页策略：发生缺页时，一次性调入多个相邻页
- 请求调页策略：发生缺页时，只调入缺页

■ 何处调入页面：外存分为文件区和对换区，对换区采用连续分配，文件区采用离散分配，对换区的磁盘I/O速度比文件区高

- 全部从对换区调入，要求进程运行前将所有文件从文件区拷贝到对换区
- 先从文件区调入；若页面被修改则换出时放入对换区，以后从对换区调入；未修改页面下次仍从文件区调入
- 未运行页面从文件区调入；运行过且被换出的页面从对换区调入

请求分页存储管理

■ 页面调入过程:

- ① 若访问页面未在内存，向**CPU**发出一缺页中断
- ② 中断处理程序首先保留**CPU**环境，分析中断原因 转入缺页中断处理程序。
- ③ 缺页中断处理程序查找页表，得到该页在外存的物理块号
- ④ 若内存能容纳新页，则启动磁盘**I/O**将缺页调入内存
- ⑤ 若内存已满，按照某种置换算法选出一页准备换出；如果该页未修改，该页不必写回磁盘；若该页已被修改，则必须写回磁盘，调到步骤4
- ⑥ 修改页表，置存在位为**1**，并将此页表项写入快表中
- ⑦ 利用修改后的页表，形成要访问数据的物理地址，读取内存数据

页面置换算法

■ 页面置换算法的设计目标

- 具有较低的页面更换频率——低缺页率
- 找出以后不再访问的页面或者较长时间不再使用的页面

■ 算法种类

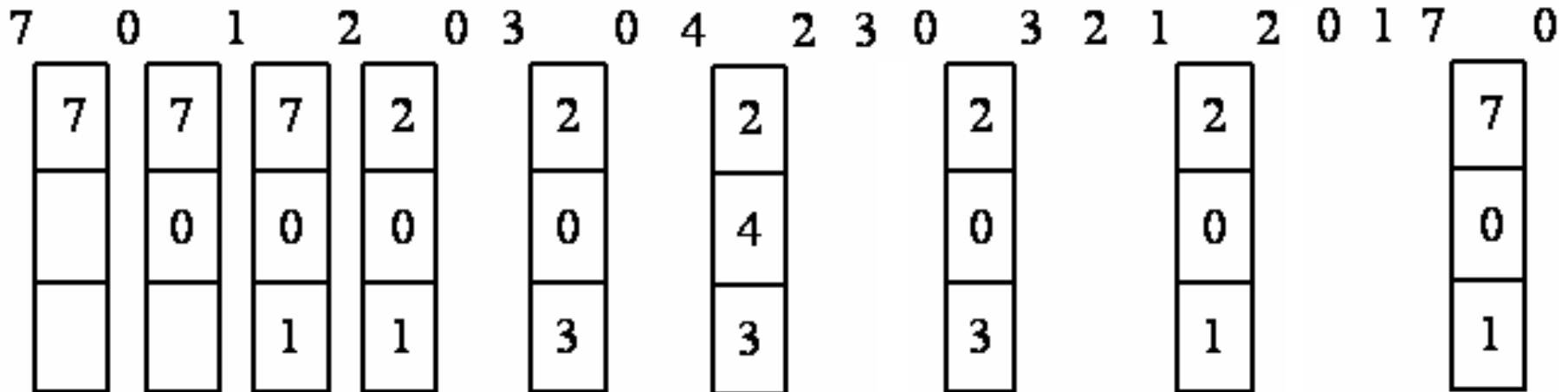
- 最佳置换算法 ·
- 先进先出算法
- LRU算法
- Clock算法
- 其他算法

页面置换算法

■ 最佳置换算法 ·

- 选择的页面：不再访问的或者较长时间不再使用
- 优点：保证最低的缺页率
- 缺点：不可能真正实现，只作为其他算法评价参考
- 特点：“往后看”，看未来，理论上的算法，不可行

引用率



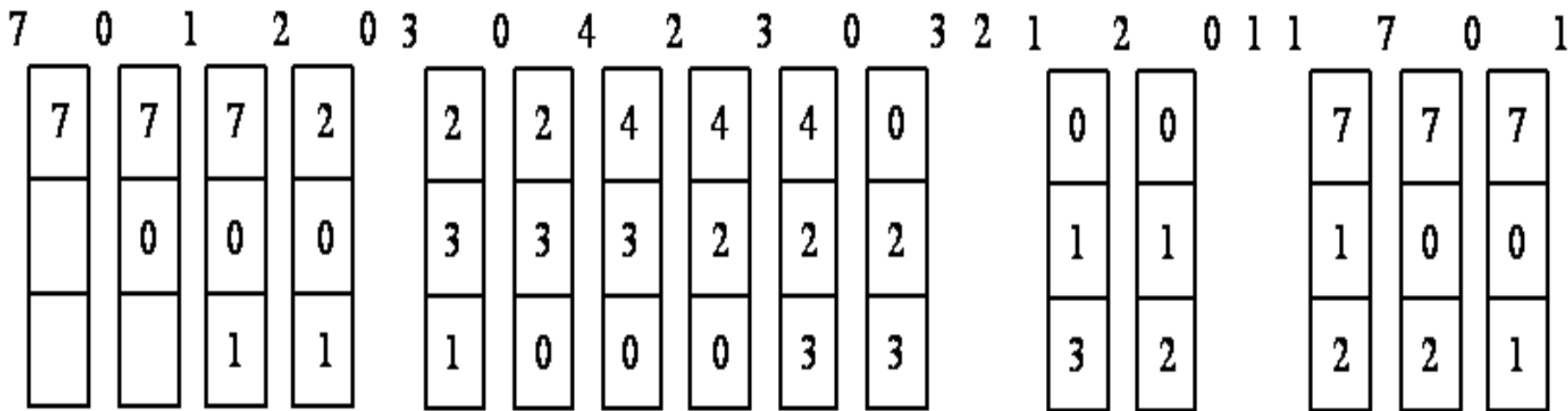
页框(物理块)

页面置换算法

■ 先进先出(FIFO)页面置换算法 ·

- ❑ 淘汰最先进入内存的页面，即选择内存驻留时间最长的
- ❑ 需要使用指针标识停留最久的位置
- ❑ 优点：算法简单；缺点：性能不佳
- ❑ 特点：只看“进入顺序”

引用率



页面置换算法

■ 最近最久未使用(LRU)置换算法

- 选择最近最久未使用的页面
- 增加访问字段记录各个页面未被访问的周期数
- 优点：性能较好；缺点：需要较多硬件支持
- 特点：“向前看”，看过去的使用情况

引用率

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

页面置换算法

- LRU置换算法需要硬件支持
- LRU硬件要解决的问题
 - 进程各个页面有多久未被访问
 - 如何快速找到最近最久未使用的页面
- LRU硬件：移位寄存器、堆栈
- 移位寄存器：
 - 每个寄存器记录进程在内存中每页的使用情况
 - 数据格式： $R=R_{n-1}R_{n-2}R_{n-3} \dots R_2R_1R_0$
 - 页面被访问一次，最高位 R_{n-1} 置1，每隔一定时间，寄存器右移一位
 - R 值最小的页面是被置换页面

页面置换算法

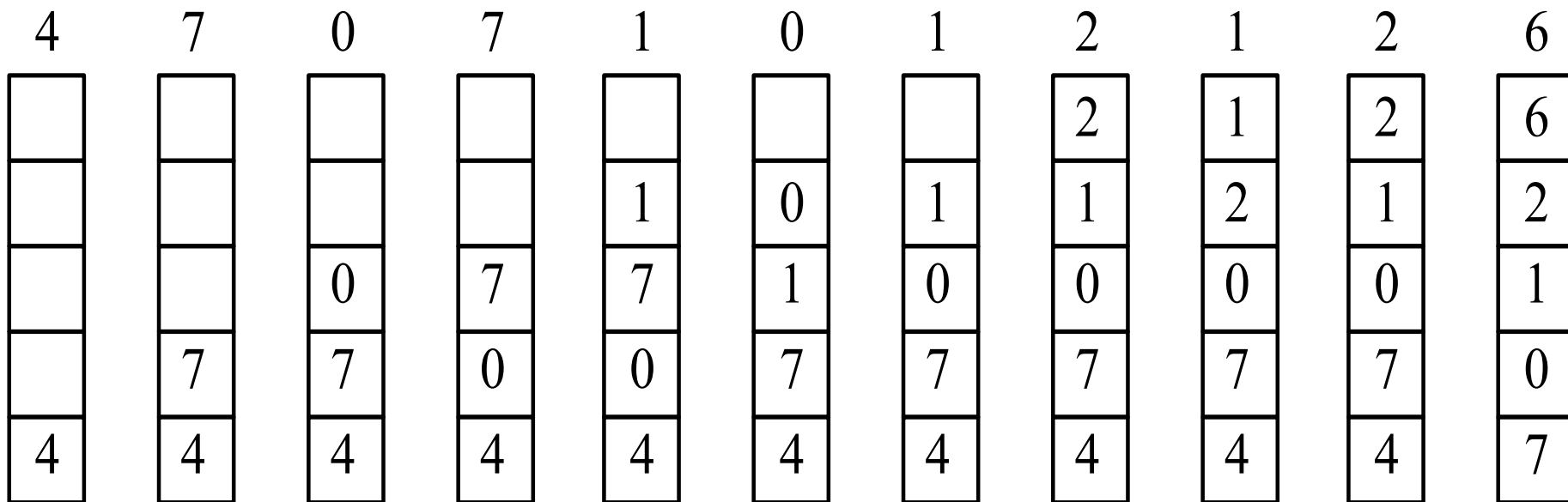
■ 移位寄存器：记录进程在内存中每页的使用情况

R 实页	R ₇	R ₆	R ₅	R ₄	R ₃	R ₂	R ₁	R ₀
1	0	1	0	1	0	0	1	0
2	1	0	1	0	1	1	0	0
3	0	0	0	0	0	1	0	0
4	0	1	1	0	1	0	1	1
5	1	1	0	1	0	1	1	0
6	0	0	1	0	1	0	1	1
7	0	0	0	0	0	1	1	1
8	0	1	1	0	1	1	0	1

页面置换算法

■ 堆栈

- 采用特殊的栈来保存当前使用的各个页面号
- 栈顶保留的是最新被访问的页
- 栈底保留的是最久未被访问的页，即置换目标

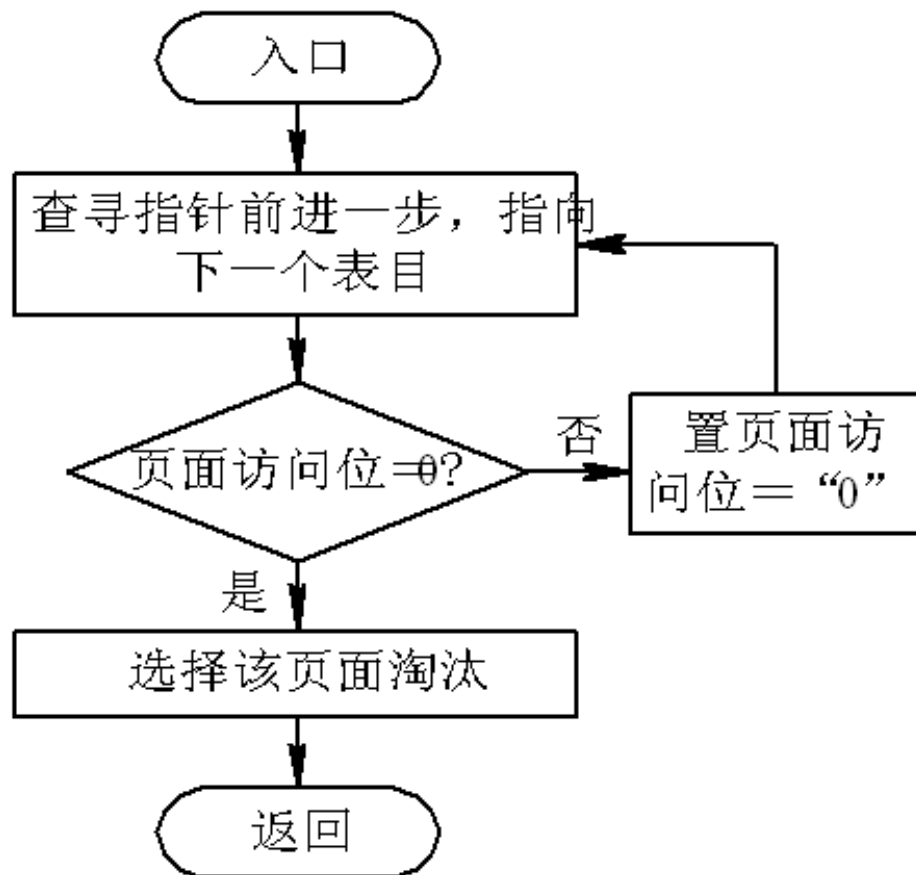


练习

- 在一个请求分页系统中，假定系统分配给一个进程的内存块数为3，进程的页面走向为1、3、1、2、4、2、1、4、5、3、1、2。若采用最近最久未使用置换算法（LRU），请画图表示页面置换过程并计算缺页次数。
- 假定开始时物理块全为空，每次调入页面都作为一次缺页处理

4.7 页面置换算法

- 简单的Clock置换算法
- LRU算法的近似算法，也称为最近未用算法。
- 为每页设置一访问位，当被访问则设置为1，未访问为0



块号	页号	访问位	指针
0			
1			
2	4	0	
3			
4	2	1	
5			
6	5	0	
7	1	1	

替换
指针

4.7 页面置换算法

■ 改进型Clock置换算法

□ 每个页面有访问位**A**和修改位**M**

□ 两个位组合成四种类型页面

1. **A=0, M=0**: 最近未被访问, 又未被修改, 最佳淘汰页
2. **A=0, M=1**: 最近未被访问, 但已被修改页。
3. **A=1, M=0**: 最近已被访问, 但未被修改。 ·
4. **A=1, M=1**: 最近已被访问且被修改, 最不应淘汰页。

4.7 页面置换算法

■ 改进型Clock的执行过程

- ① 从当前指针位置开始扫描循环队列，寻找第**1**类页面，不改变访问位**A**，将遇到的第一个这类页面作为淘汰页
- ② 第一步查找失败，寻找第**2**类页面，将遇到的第一个这类页面作为淘汰页。在扫描期间，所有扫描过的页面的访问位**A**都置**0**。
 - 所有**3**类页面变成**1**类页面，**4**类页面变成**2**类页面
- ③ 第二步失败，重复第一步
- ④ 如果仍失败，再重复第二步，此时就一定能找到被淘汰的页。

4.7 页面置换算法

■ 改进型Clock置换算法举例

块号	页号	访问位	修改位	指针
0				
1				
2	4	1	1	
3				
4	2	1	0	
5				
6	5	1	1	
7	1	1	0	

替换指针

块号	页号	访问位	修改位	指针
0				
1				
2	4	0	1	
3				
4	2	0	0	
5				
6	5	0	1	
7	1	0	0	

替换指针

- 优点：减少了磁盘的I/O操作次数
- 缺点：可能需要几轮扫描，增加了系统开销
- 特点：使用两个指标判断，访问位+修改位

练习

- 在一个请求分页系统中，已知某进程有4页在内存，页码依次为12，23，31，48，其A、M位分别为（1，1）、（1，0）、（1，0）、（1，1），若采用改进型Clock置换算法，选择哪页淘汰？选择完成后，四个页的A、M位分别为多少？

页面置换算法

■ 其它置换算法

□ 最少使用(LFU: **Least Frequently Used**)置换算法

- 对每个页面设置一个字段（移位寄存器），用来记录页面被访问的频率

□ 页面缓冲算法

- 采用可变分配和局部置换方式
- 当一个进程换进换出频率很低时，选择页面淘汰，以备其他进程使用
- 被淘汰页面若发生修改，放入已修改链表，否则放入空闲链表
- 修改链表的页面积累一定数量，一次性写回硬盘

请求分段存储管理方式

■ 请求分段中的硬件支持:

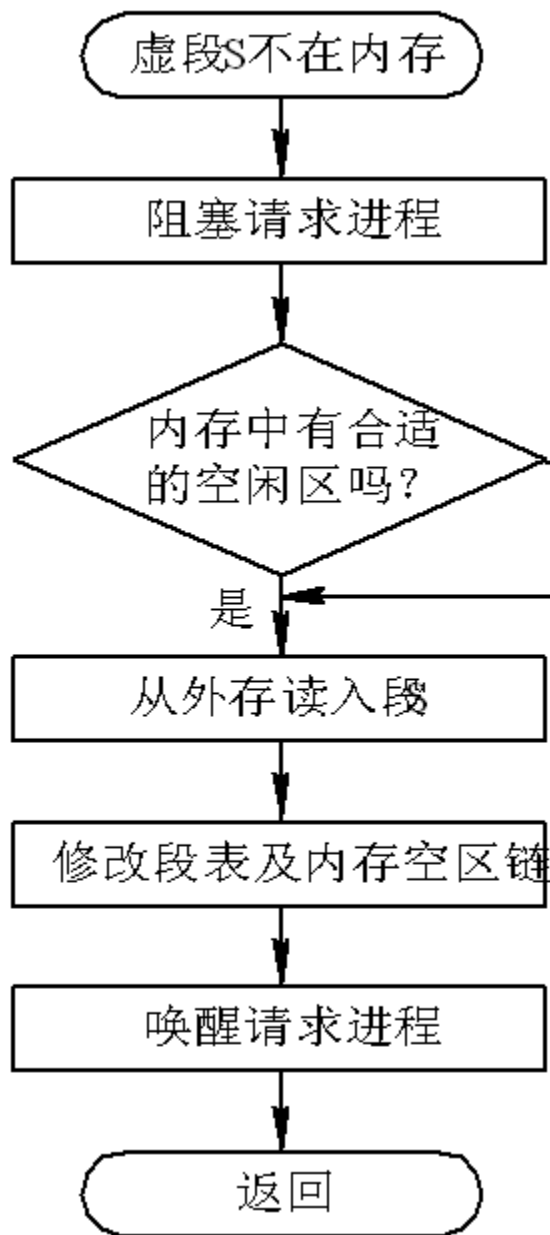
☐ 段表机制、缺段中断机构、地址变换机构

■ 段表机制

段名	段长	段的基址	存取方式	访问字段	修改位	存在位	增补位	外存始址
----	----	------	------	------	-----	-----	-----	------

- ☐ 段名、段长、基址：原有
- ☐ 存取方式：执行、只读、读/写
- ☐ 访问字段：一段时间内访问次数
- ☐ 修改字段：调入后是被修改过
- ☐ 存在位：是否已在内存
- ☐ 增补位：是否做过动态增长
- ☐ 外存地址：本段在外存的起始盘块号

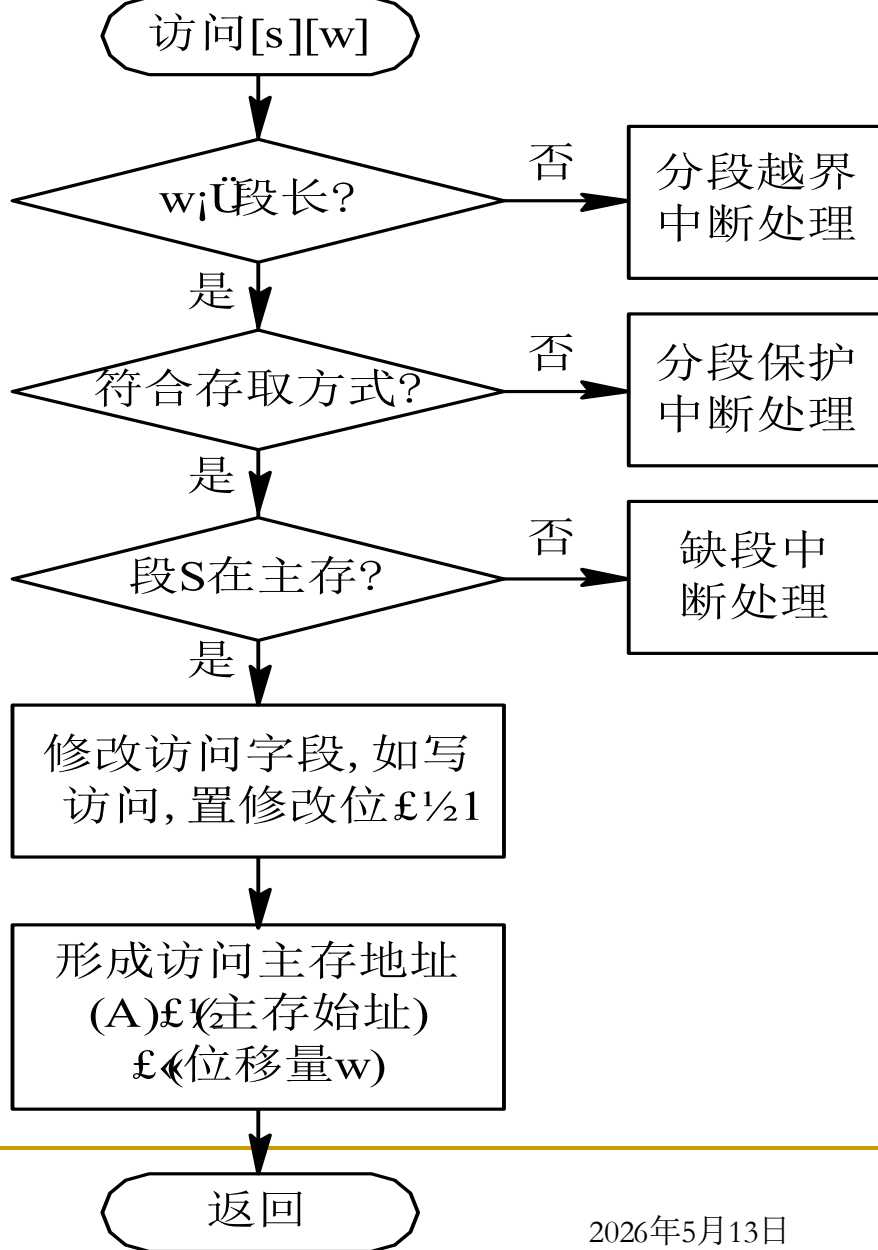
请求分段存储管理方式



■ 缺段中断机构

- ❑ 一条指令可能发生多次缺段中断
- ❑ 整条指令只会在一个段内
- ❑ 整个数据只会在一个段内

请求分段存储管理方式



地址变换机构

第四章复习

- 存储器层次结构：CPU寄存器、主存、辅存
- 连续分配：分配的内存空间是连续的，非离散的
 - 四种分配方式、动态分配三种算法：FF、CF、BF
- 分页管理：页面、物理块
 - 逻辑地址转换为物理地址的计算
 - 多级页表的计算
- 分段管理：分段是具有逻辑意义的分隔
 - 分段的作用、分页与分段的区别
 - 段页式管理：先分段后分页，地址计算
- 虚拟存储器：请求调入和置换功能，逻辑上扩充
 - 定义、三个特征
- 分页请求管理：基本分页上增加调入和置换功能，硬件设备
- 页面置换算法：最佳置换、先进先出、LRU、改进型Clock
- 请求分段管理：基本分段上增加调入和置换功能，硬件设备